

Compiladores — Análise Sintática

Análise Léxica + Sintática com ANTLR 3 — Subconjunto de C

14.06.2026

Jéssica Pereira, Kauã Bispo, Kenandja Krishna, Liriel Gomes, Lucas Ramos

1	Introdução	3
2	O Gerador ANTLR	8
3	A Gramática	11
4	Análise na Prática	19
5	Conclusão	25

1 Introdução

Sobre este Trabalho

4 / 27

Disciplina: Compiladores

Professor: Alexandre Paes

Instituição: Universidade Federal de Alagoas — Campus Arapiraca

Equipe:

1. Jéssica Pereira
2. Kauã Bispo
3. Kenandja Krishna
4. Liriel Gomes
5. Lucas Ramos

Gerador utilizado: ANTLR 3

Linguagem alvo: Java

Gramática: `CParser.g` — Lexer + Parser para um subconjunto de C (Soma dos Pares)

“A análise sintática, também conhecida como *parser*, é a **segunda fase** de um compilador. Ela recebe a sequência de tokens produzida pelo analisador léxico e verifica se essa sequência está de acordo com as **regras gramaticais** da linguagem, construindo uma **árvore sintática** que representa a estrutura hierárquica do programa.”

— Johny Douglas — *Compiladores para Humanos*

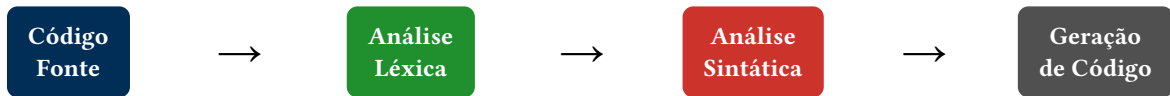
O que é Análise Sintática? (ii)

6 / 27

O parser consome os tokens do lexer e reconhece a estrutura do programa:

```
PROGRAM
├─ INCLUDE_DIR
│   └─ 'stdio'
└─ FUNC_DEF
    ├─ INT 'int'
    ├─ ID 'main'
    └─ BLOCK
        ├─ VAR_DECL ...
        ├─ FOR_STMT ...
        └─ RETURN_STMT ...
```

O resultado é uma **AST (Abstract Syntax Tree)** — árvore sintática abstrata.



A **análise sintática** é a **segunda etapa**: consome os tokens do lexer e constrói a **AST** que representa a estrutura do programa.

2 O Gerador ANTLR

O que é o ANTLR?

ANTLR — *Another Tool for Language Recognition*

Criado por **Terence Parr** (Universidade de São Francisco)

- Ferramenta de geração de **Lexers** e **Parsers** a partir de uma gramática **.g**
- Amplamente utilizado na construção de linguagens de programação, frameworks e ferramentas de análise
- Suporta múltiplas linguagens alvo: Java, Python, C, C#, C++, PHP

Neste trabalho, geramos o código em **Java**.

Como o ANTLR Funciona

Você escreve:

- Um arquivo `NomeDaGramatica.g`
- Regras do **Lexer** (MAIÚSCULAS)
- Regras do **Parser** (minúsculas)
- Opção de linguagem alvo e saída AST

Executa:

```
antlr3 CParser.g
```

O ANTLR gera automaticamente:

- `CParserLexer.java` — lexer gerado
- `CParserParser.java` — parser gerado
- `CParser.tokens` — tabela de tokens

O `grammar CParser` com `output = AST` instrui o ANTLR a gerar o parser com suporte a árvores sintáticas usando as `rewrite rules (->)`.

3 A Gramática

A gramática `CParser.g` define **lexer** + **parser** num único arquivo:

Tipo: `grammar CParser` — gramática combinada (lexer + parser)

Saída: `output = AST` — ANTLR gera suporte a árvores sintáticas

Tokens imaginários: Nós da AST sem correspondência no texto fonte

Rewrite rules: Sintaxe `-> ^(NÓ filhos...)` que constrói a AST

Tokens imaginários (nós da AST):

`PROGRAM`

`FUNC_DEF`

`BLOCK`

`VAR_DECL`

`IF_STMT`

`RETURN_STMT`

`FUNC_CALL`

`INCLUDE_DIR`

`FOR_STMT`

`FOR_INIT`

`FOR_COND`

`FOR_UPDATE`

`ASSIGN_STMT`

`EXPR_STMT`

`ARG_LIST`

```
program
: includeDir functionDef EOF
-> ^(PROGRAM includeDir functionDef)
;

includeDir
: INCLUDE LT ID (DOT ID)? GT
-> ^(INCLUDE_DIR INCLUDE ID+)
;

functionDef
: type ID LPAREN RPAREN block
-> ^(FUNC_DEF type ID block)
;

block
: LBRACE statement* RBRACE
```

```
    -> ^(BLOCK statement*)
    ;

statement
: varDecl | assignStmt | forStmt
| ifStmt | returnStmt | exprStmt
;

varDecl
: type ID (ASSIGN expr)? SEMI
-> ^(VAR_DECL type ID expr?)
;

assignStmt
: ID ASSIGN expr SEMI
-> ^(ASSIGN_STMT ID expr)
;
```

```
forStmt
: FOR LPAREN forInit SEMI
  forCond SEMI forUpdate RPAREN block
-> ^(FOR_STMT forInit forCond
      forUpdate block)
;

ifStmt
: IF LPAREN expr RPAREN block
-> ^(IF_STMT expr block)
;

returnStmt
: RETURN expr SEMI
-> ^(RETURN_STMT expr)
;
```

```
expr
: term ((PLUS|EQ|LE|LT|GT|MOD) term)*
;

term
: ID LPAREN argList? RPAREN
  -> ^(FUNC_CALL ID argList?)
| ID | NUM | STRING
;
```


Tabela de Tokens

17 / 27

Token	Tipo	Descrição
INT, FOR, IF, RETURN, INCLUDE	Literal	Palavras-chave individuais
STRING	Regular	'"' (~'"')* '"' — strings entre aspas (ex.: "%d")
ID	Regular	('a'..'z' 'A'..'Z' '_') ('a'..'z' 'A'..'Z' '0'..'9' '_')*
NUM	Regular	'0'..'9'+
EQ, LE, INC	Literal	Operadores multi-caractere: ==, <=, ++
PLUS, ASSIGN, MOD, DOT	Literal	Operadores: +, =, %, .
LPAREN, RPAREN, LBRACE, RBRACE, SEMI, LT, GT, COMMA	Literal	Delimitadores e pontuação

Tabela de Tokens (ii)

18 / 27

Token	Tipo	Descrição
WS	Regular	Espaços/quebras — descartados via <code>skip()</code>
<code>COMMENT</code>	Regular	Comentários de linha <code>//</code> — descartados via <code>skip()</code>
EOF	Implícito	Fim da entrada

4 Análise na Prática

O Arquivo de Gramática (.g)

O arquivo `CParser.g` combina lexer e parser:

- `grammar CParser` — gramática combinada
- `output = AST` — ativa geração de árvore
- Tokens imaginários declarados em `tokens { }`
- Regras do **parser** em minúsculas com rewrite rules `-> ^( ... )`
- Regras do **lexer** em MAIÚSCULAS (idênticas ao `CLexer.g`)

Para gerar e executar:

```
antlr3 CParser.g
javac -cp antlr-3.x.jar \
    CParserLexer.java \
    CParserParser.java \
    TestParser.java
java -cp .:antlr-3.x.jar TestParser
```

```
grammar CParser;

options {
    language = Java; output = AST;
}

tokens {
    PROGRAM; FUNC_DEF; BLOCK; VAR_DECL; FOR_STMT;
    FOR_INIT; FOR_COND; FOR_UPDATE; IF_STMT;
    RETURN_STMT; ASSIGN_STMT; EXPR_STMT; FUNC_CALL;
    ARG_LIST; INCLUDE_DIR;
}

program
    : includeDir functionDef EOF
    -> ^(PROGRAM includeDir functionDef)
    ;

// ... demais regras do parser ...

// Regras do lexer (mesmas do CLexer.g)
```

Passo	Componente	O que faz
1	Input Stream	<code>ANTLRStringStream(code)</code> — encapsula a string caractere a caractere
2	Lexer (AFD)	<code>CParserLexer(input)</code> — tokeniza a entrada
3	Token Stream	<code>CommonTokenStream(lexer)</code> — coleta tokens em fila para o parser
4	Parser	<code>CParserParser(tokens)</code> — consome os tokens e valida a gramática
5	Regra inicial	<code>parser.program()</code> — dispara a análise a partir de <code>program</code>
6	AST	<code>result.getTree()</code> — retorna a árvore sintática construída
7	Saída	Imprime tokens (fase léxica) + AST hierárquica (fase sintática)

Fase 1 do `TestParser.java`: mesma tokenização do projeto anterior.

Linha	Coluna	Tipo	Valor
1	1	INCLUDE	'#include'
1	9	LT	'<'
1	10	ID	'stdio'
1	15	DOT	'.'
1	16	ID	'h'
1	17	GT	'>'
3	1	INT	'int'
3	5	ID	'main'
3	9	LPAREN	'('
3	10	RPAREN	')'
3	12	LBRACE	'{'
4	5	INT	'int'
4	9	ID	'soma'

Linha	Coluna	Tipo	Valor
4	14	ASSIGN	'='
4	16	NUM	'0'
4	17	SEMI	';'
5	5	FOR	'for'
5	9	LPAREN	'('
5	10	INT	'int'
5	14	ID	'i'
5	16	ASSIGN	'='
5	18	NUM	'0'
5	19	SEMI	';'
...			
Total de tokens: 62			

Fase 2: o parser valida a estrutura e constrói a **AST**.

Resultado da validação:

- ✓ Programa ACEITO pela gramática!
Nenhum erro sintático encontrado.

AST — formato linear:

```
(PROGRAM
  (INCLUDE_DIR #include stdio h)
  (FUNC_DEF int main
    (BLOCK
      (VAR_DECL int soma 0)
      (FOR_STMT ...)
      (EXPR_STMT ...)
      (RETURN_STMT 0))))
```

AST — formato hierárquico:

```
PROGRAM
├─ INCLUDE_DIR
│   ├── INCLUDE '#include'
│   ├── ID 'stdio'
│   └─ ID 'h'
└─ FUNC_DEF
    ├── INT 'int'
    ├── ID 'main'
    └─ BLOCK
        ├── VAR_DECL
        │   ├── INT 'int'
        │   ├── ID 'soma'
        │   └─ NUM '0'
        └─
```

```
├ FOR_STMT
│   ├── FOR_INIT ...
│   ├── FOR_COND ...
│   ├── FOR_UPDATE ...
│   └ BLOCK
│       └ IF_STMT ...
├ EXPR_STMT
│   └ FUNC_CALL
│       ├── ID 'printf'
│       └ ARG_LIST ...
└ RETURN_STMT
    └ NUM '0'
```


5 Conclusão

O que aprendemos:

- A **análise sintática** é a 2ª fase da compilação
- O parser consome tokens e verifica conformidade com a gramática
- Rewrite rules ($\rightarrow \wedge(\dots)$) constroem a **AST** automaticamente
- Tokens imaginários (PROGRAM, FUNC_DEF, etc.) formam os nós internos da árvore
- Um programa **aceito** produz uma AST; um **rejeitado** gera erros sintáticos

Ferramentas utilizadas:

- ANTLR 3
- Runtime Java do ANTLR 3
- javac / java (JDK)

Regras do parser implementadas:

- `program` — estrutura completa do arquivo C
- `includeDir` — diretiva `#include`
- `functionDef` — definição de função
- `block` — bloco de instruções `{ ... }`
- `varDecl` — declaração de variável
- `assignStmt` — atribuição
- `forStmt` — laço `for` com `forInit`, `forCond`, `forUpdate`
- `ifStmt` — condicional `if`
- `returnStmt` — instrução `return`
- `exprStmt` — expressões como `printf(...)`
- `expr` / `term` / `argList` — expressões e argumentos

- **DOUGLAS, Johnny.** *Compiladores para Humanos*. GitBook. Disponível em: johnidouglas.gitbook.io
- **ANTLR.** *ANTLR 3 Documentation*. Disponível em: antlr3.org
- **PARR, Terence.** *The Definitive ANTLR Reference*. Pragmatic Bookshelf, 2007.
- **Repositório do projeto:** disponível na descrição do vídeo (GitHub)